⚙️ Manual del Programador (Técnico) - LabManager V2.0

1. Introducción

Este documento detalla rigurosamente la ingeniería detrás de la plataforma de nueva generación **LabManager**. Está elaborado para que otros desarrolladores, ingenieros de software, o auditores de TI puedan mantener, clonar, depurar o expandir la aplicación a nuevas fases sin pérdida de la lógica fundamental. Aborda arquitectura, estructura de código fuente, y la topología de la base de datos distribuida.

---

2. Visión Arquitectónica del Sistema

El sistema respeta una **Arquitectura en Capas Invertida Controlada (Cliente-Servidor)**:

1. **Top-Level (Frontend):** *Single Page Application (SPA)* construida sobre el DOM virtual (React). Totalmente desacoplada lógica e infraestructuralmente (en dominios cruzados).
2. **Capa Middleware (Gateways y WS):** Conexiones HTTP asíncronas vía capa `Axios` para transacciones y `SockJS/STOMP` persistentes bajo protocolo de túnel *ws://* para emisiones vivas (Broadcasts).
3. **Capa Negocio (Backend Core):** Una aplicación autónoma de Java Spring montada en base a beans (Dependency Injection) exponiendo Endpoints REST completos.
4. **Capa Inversión de Datos (ORM):** Hibernate embebido traduce lógica POJO Java (Entidades) directamente a lenguaje Data Definition (DDL) y manipula en lenguaje SQL abstracto de Dialectos agresivos en la nube PostgreSQL.

---

3. Stack Tecnológico de Referencia

3.1 Nivel Backend (Java App)

- **Base:** Java JDK 17 (Criterio para Record Types y Patrones de Switch actuales).
- **Framework:** Spring Boot Version 3.2.0.
- **Seguridad (Auth):** Filtros Spring Security 6 encadenados y autorizaciones vía tokens `jjwt-api` sin estados de sesión web clásica de TomCat.
- **Tiempo Real:** `spring-boot-starter-websocket` anclado a un *MessageBrokerRegistry* local mapeado al canal `/topic`.
- **Motor PDF/Excéntrico:** `com.itextpdf.itext7-core` y dependencias transitorias de `org.apache.poi` para parseo sintáctico de árboles XML para Microsoft Excel (`.xlsx`).

3.2 Nivel Frontend (Vite App)

- **Transpilador:** Typescript estricto sobre Motor Node.js v18+.
- **Motor Build:** ViteJS. Enormemente superior al obsoleto CRA de Webpack; asegura un *Hot Module Reload* atómico instantáneo.
- **Lógica UI:** React 18, React Router DOM (v6+), `lucide-react` para carga nativa vectorizada (SVG) iconográfica.
- **Hardware Interoperacional:** `@yudiel/react-qr-scanner`, encapsula llamadas al `navigator.mediaDevices` nativo del navegador abriendo cámaras multi-lentes.

---

4. Estructura de Directorios

Módulo Backend (`/src/main/java/com/university/labmanager/`)

- **`config/` :** Define el CORS absoluto (vía `WebSecurityConfig.java`) y el `WebSocketConfig.java` inyectando endpoints púbicos sin re-autenticar STOMP, confiando en cruces de Token Header (`/ws`).
- **`controller/` :** Controladores anotados típicamente con `@RestController` y mapas Base `/api/v1/X` .
- **`model/` :** POJOS con Anotaciones JPA típicas (`@Entity` , `@Table`) y directivas de persistencia `@JsonBackReference` y `@JsonManagedReference` vitales para evadir referencias o lazos infinitos `Jackson Serializable Error` al momento de compilar JSONS hacia React.
- **`repository/` :** Exclusivamente código `extends JpaRepository` y de muy alto valor avanzado `JpaSpecificationExecutor` (Solución radical contra fallos nativos de NullPointers PostgreSQL).
- **`service/` :** Envolturas de métodos transaccionales (`@Transactional`). Solo aquí descansa la lógica dura del negocio (Por norma de diseño, al Controller no se le incrustaron sentencias `if-else` nativas).
- **`dto/` :** Modelos anidables vacíos (Beans Planos). Ejemplo: `LaptopHistoryDTO` para mandar un solo string JSON en vez de 3 queries SQL.

Módulo Frontend (`/labmanager-frontend/src/`)

- **`components/` :** Fragmentos lógicos de nivel atómico (`Button.tsx` , `Chatbot.tsx` , `NotificationDropdown.tsx`).
- **`pages/` :** Enrutados base: `AdminDashboard.tsx` es una macro-vista monstruo segmentada internamente en Sub-Tags reactivas de estado puro.
- **`hooks/` :** Controladores abstractos. Elemento valioso `useWebSocket.ts` , re-ejecutado y memorizado solo si cambia su URL base en modo producción vs modo local (Desactible mediante *API_URL* de `api.ts`).

---

5. Profundizando Componentes Sofisticados de Código

A) Implementación de Reportes (JPA Specifications)

Originalmente, construcciones JPQL duras mediante `@Query(...)` desencadenaron problemas en PostgreSQL al buscar fechas híbridas en la nube originando *HTTP 500 Internals*. La solución programada fue reescribir hacia el sub-lenguaje de **Criteria Builder**: Se diseñan clases predictivas `Specification<Reservation> spec = ...` con sentencias lógicas donde si el Filtro es *Nulo*, sencillamente no se añade dinámicamente al SQL final compilado (`Predicate` builder de EntityManager).

B) Mensajería WebSockets Reactiva

No utilizamos librerías pesadas como *Socket.io*. Optamos por STOMP puro de Spring.

- El servidor enlista su endpoint público en `/ws` .
- Se implementó el inyector `SimpMessagingTemplate` dentro de nuestro `NotificationService` . Cuando ocurre el método `createNotification` , se manda forzosamente el canal de túnel `"/user/" + destUserId + "/queue/notifications"` .
- En React, `@stomp/stompjs` suscriben un hilo oculto de subproceso DOM consumiendo el evento `onConnect` . Si el hook atrapa nueva carga útil, dispara inmediatamente un recálculo estatal (El componente campana de alerta eleva su contador sin alterar los nodos del padre).

C) Escaneo QR y Media Devices API

El escaneo de hardware de frontend usa variables de entorno reactivas temporales en el Dom para prevenir *Memory-Leaks* (Pérdidas de memoria RAM en Firefox/Chrome) al no apagar adecuadamente el led/sensor físico de cámara tras escanear un QR. Todo flujo se deshabilita desde la función `useEffect cleanup rutine` .

---

6. Base de Datos Cruda (DBA) y Despliegue CI/CD

El sistema requiere PostgreSQL ^14.0. Se hostea comúnmente en un Cloud Pooling (Supabase / DigitalOcean Databases) asumiendo los puertos fijos remotos `5432` o `6543`.

- **Propiedades y Variables Críticas del Sistema (`application.properties`):**
  - `spring.datasource.url` debe setear el connection pooling correcto: `jdbc:postgresql://<HOST>:<PORT>/postgres?user=...`
  - `spring.jpa.hibernate.ddl-auto=update` : Auto-esquematiza las relaciones uno a n, creando llaves de enlace intermedias automáticamente sin programador DBA al momento que reinicie el servidor de Digital Ocean (Ajustable a `validate` en modo ultra-restrictivo).
- **Variables Cliente Frontend (`.env`):** El cliente obedece a un `axiosBaseUrl` y un conector WsUrl apuntados a un enrutador inverso que mapea subdirectorios de puerto.

El compilado final se unifica utilizando Maven `clean install -DskipTests` y NodeJS se aísla empacando en `/dist` donde todo el TSX de cientos de módulos React termina compactado brutalmente a solo ~3-4 archivos Javascript unificables ultra-ligeros con minificación nativa.

¿Necesitas Modificar el Código Base? Clona el repositorio desde GitHub, levanta el Docker `postgres` de forma local, configura tus URL en puertos `localhost:8080` (Para Spring) y arranca ViteJS. Las actualizaciones de TypeScript estarán activas al presionar *Guardar*.